



Програмиране за мобилни интернет устройства

ИЗГРАЖДАНЕ НА ГРАФИЧЕН ИНТЕРФЕЙС С
КОТЛИН. JETPACK COMPOSE

Недостатъци на Класическия подход

- ▶ Сложност и четимост на кода
- ▶ Трудности при управление на състоянието
- ▶ Зависимост от XML и Activity/Fragment код
- ▶ Трудности при използване на динамични UI компоненти
 - ▶ много различни View-и и адаптери
- ▶ Ограничени възможности за повторно използване на компоненти

Jetpack Compose

- ▶ позволява създаване на UI компоненти директно чрез код - декларативен подход
- ▶ премахва нуждата от XML файлове и улеснява управлението на състояния
- ▶ Поддържа **State** - улеснява обновяването на UI при промени в данните

Jetpack Compose

- ▶ декларативното програмиране - позволява да се определи как трябва да изглежда UI при дадено състояние, вместо да се пишат конкретни инструкции за това как да се нарисува и подреди всеки елемент
 - ▶ Вместо да обновяваме даден `TextView` при промяна на текст, в Compose просто задаваме новото състояние, а интерфейсът автоматично се актуализира

Jetpack Compose

```
@Composable
fun CounterExample() {
    var count by remember { mutableStateOf(0) }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            text = "Count: $count",
            fontSize = 24.sp,
            modifier = Modifier.padding(bottom = 16.dp)
        )

        Button(onClick = { count++ }) {
            Text("Increase Count")
        }
    }
}

@Preview(showBackground = true)
@Composable
fun PreviewCounterExample() {
    CounterExample()
}
```

@Composable

@Composable

```
fun Greeting(name: String) {  
    Text(text = "Hello, $name!")  
}
```

- ▶ @Composable - специфична за Jetpack Compose
 - ▶ позволява на функцията да участва в рендирането на интерфейса, като информира Compose да се отнася към нея като част от UI дървото.

Интегриране с останалата част от Android

- ▶ Jetpack Compose е проектиран да работи безпроблемно с традиционните Android компоненти като View и Fragment.
- ▶ Съществува **ComposeView**, който позволява вмъкване на Compose UI в съществуващите XML Layout-и, както и използването на традиционните View компоненти в Jetpack Compose

ОСНОВНИ КОМПОНЕНТИ В Jetpack Compose

► Text

- Използва се за показване на текстови елементи.

► @Composable

```
fun GreetingText() {  
    Text(text = "Hello, Compose!", fontSize = 20.sp, color = Color.Blue)  
}
```

► Image

► @Composable

```
fun ProfileImage() {  
    Image(painter = painterResource(R.drawable.profile_picture),  
        contentDescription = "Profile Image")  
}
```


ОСНОВНИ КОМПОНЕНТИ В Jetpack Compose

▶ Button

- ▶ Класически бутон, който поддържа **onClick** за интеракции
- ▶ @Composable

```
fun ClickableButton() {  
    Button(onClick = { /* действие при клик */ }) {  
        Text("Click Me")  
    }  
}
```

Row и Column

- ▶ Използват се за подреждане на елементи съответно по хоризонтала и вертикала
- ▶ Предоставят гъвкавост за организиране на елементи по лесен и интуитивен начин

- ▶ @Composable

```
fun LayoutExample() {  
    Column(modifier = Modifier.padding(16.dp)) {  
        Text("Item 1")  
        Text("Item 2")  
        Text("Item 3")  
    }  
}
```

Scaffold

- ▶ Scaffold е контейнер за структура на екрани с елементи като TopAppBar, BottomNavigation, FloatingActionButton, и Drawer

- ▶ @Composable

```
fun MainScreen() {  
    Scaffold(  
        topBar = { TopAppBar(title = { Text("My App") }) },  
        floatingActionButton = {  
            FloatingActionButton(onClick = { /* действие */ }) {  
                Icon(Icons.Filled.Add, contentDescription = "Add")  
            }  
        }  
    ) { padding ->  
        Content(modifier = Modifier.padding(padding))  
    }  
}
```

Модификатори (Modifiers)

- ▶ Модификаторите в Jetpack Compose са инструмент за добавяне на стилизиране и функционалност към Composable елементите. Например:
 - ▶ `Padding`: Добавя вътрешен отстъп около елемент.
 - ▶ `Background`: Променя фона на елемент.
 - ▶ `Size`: Определя размера на елемента.
 - ▶ `clickable`: Позволява да добавим клик събитие към елемент.

Модификатори (Modifiers)

@Composable

```
fun StyledText() {  
    Text(  
        text = "Hello, Jetpack Compose!",  
        modifier = Modifier  
            .padding(16.dp)  
            .background(Color.LightGray)  
            .fillMaxWidth()  
            .padding(8.dp),  
        color = Color.Black  
    )  
}
```

Управление на цветовете в Jetpack Compose

```
@Composable
fun ColoredText() {
    Text(
        text = "Colored Text",
        color = Color.Blue,
        modifier = Modifier.padding(16.dp)
    )
}
```

Оформление с Card

@Composable

```
fun CardExample() {
```

```
    Card(elevation = 4.dp, modifier = Modifier.padding(8.dp)) {
```

```
        Text("This is inside a card")
```

```
    }
```

```
}
```

Анимации в Jetpack Compose

- ▶ Поддържа лесно използване на анимации, като анимиране на размери, позиции и цветове.
- ▶ Някои основни функции за анимации включват `animateDpAsState`, `animateColorAsState`, и `Crossfade`.
- ▶ `@Composable`

```
fun AnimatedBox() {  
    var expanded by remember { mutableStateOf(false) }  
    val size by animateDpAsState(targetValue = if (expanded) 200.dp else 100.dp)  
  
    Box(  
        modifier = Modifier  
            .size(size)  
            .background(Color.Blue)  
            .clickable { expanded = !expanded }  
    )  
}
```


Предварителен преглед (Preview) на Composable функции

- ▶ предварителен преглед на Composable функциите в Android Studio чрез анотацията @Preview.
- ▶ Това улеснява разработчиците да видят как ще изглежда интерфейсьт, без да стартират приложението на емулятор
- ▶ @Preview

```
@Composable
```

```
fun PreviewGreeting() {
```

```
    Greeting(name = "Compose")
```

```
}
```

Jetpack Compose

- ▶ Compose наблюдава промените в състоянието и автоматично актуализира само тези части на интерфейса, които са пряко засегнати от промяната

Управление на състоянието (State Management)

- ▶ В декларативния подход, при промяна на състоянието (например текст или стойност на брояч), Jetpack Compose автоматично обновява свързаните UI компоненти.
- ▶ **MutableState** и **remember** се използват за управление на състоянието в Composable функции.

- ▶ @Composable

```
fun Counter() {  
    var count by remember { mutableStateOf(0) }  
    Text(text = "Counter: $count")  
    Button(onClick = { count++ }) {  
        Text("Increase")  
    }  
}
```

Сложни Layout-и с ConstraintLayout

► Подобен на XML-базирания ConstraintLayout

► @Composable

```
fun ComplexLayout() {  
    ConstraintLayout {  
        val (button, text) = createRefs()  
  
        Button(onClick = { /* TODO */ }, modifier = Modifier.constrainAs(button) {  
            top.linkTo(parent.top) }) {  
            Text("Button")  
        }  
  
        Text("Hello", Modifier.constrainAs(text) { top.linkTo(button.bottom) })  
    }  
}
```